Contributing

Check out the Contributing Guide for more details on how to contribute. For guidelines on how to be a helper or teacher, check out the For teachers page. As always, make sure to adhere to the Code of Conduct.

:bug: Issues and bugs

The easiest way to contribute is to report issues or bugs that you might find while using or reading the workshop material. You can do this by creating a new issue on our GitHub repository.

:pencil2: Adding or modifying content

To contribute to the workshop, you first need to install uv and justfile. We use uv and justfile to manage our project, such as to run checks and build the workshop website. Both the uv and justfile websites have a more detailed guide on using uv, but below are some simple instructions to get you started.

It's easiest to install uv first and then install justfile using uv. Then, to install justfile, run:

```
uv tool install rust-just
```

We keep all our development workflows in the `justfile`, so you can explore it to see what commands are available. To see a list of commands available, run:

```
just
```

As you contribute, make sure your changes will pass our checks by opening a terminal so that the working directory is the root of this workshop's parent/root folder and running:

```
just run-all
```

When committing changes, please try to follow Conventional Commits as Git messages. Using this convention allows us to be able to automatically create a release based on the commit message by using Cocogitto. If you don't use Conventional Commits when making a commit, we will revise the pull request title to follow that format. That's because we use squash merges when merging pull requests, so all other commits in the pull request will be squashed into one commit.

:file_folder: Support and infrastructure files

- `.copier-answers.yml`: Contains the answers you gave when copying the project from the template. **You should not modify this file directly.**
- `.gitignore`: This ignore file tells Git which files to not track. Unless you know what you are doing, it's best to not touch this file.
- `_quarto.yml`: Quarto configuration file for the website, including settings for the website, such as the theme, navigation, and other options.
- `_metadata.yml`: Quarto metadata file for the website, including information about the project, such as the titles and GitHub names.
- `.pre-commit-config.yaml`: Pre-commit configuration file for managing and running checks before each commit.
- `.config/`: Contains configuration files for various tools used in the project, such as:
  - `typos.toml`: typos spell checker configuration file.
  - `rumdl.toml` and `panache.toml`: rumdl and Panache configuration file for formatting Markdown files in the project.
- `.zenodo.json`: Structured citation metadata for your project when archived on Zenodo. This is used to add the metadata to Zenodo when a GitHub release has been uploaded to Zenodo.
- `justfile`: `just` configuration file for scripting project tasks.

- `.editorconfig`: Editor configuration file for EditorConfig to maintain consistent coding styles across different editors and IDEs.
- `CONTRIBUTING.md`: Guidelines for contributing to the project.
- `.github/`: Contains GitHub-specific files, such as issue and pull request templates, workflows, dependabot configuration, pull request templates, and a CODEOWNERS file.